

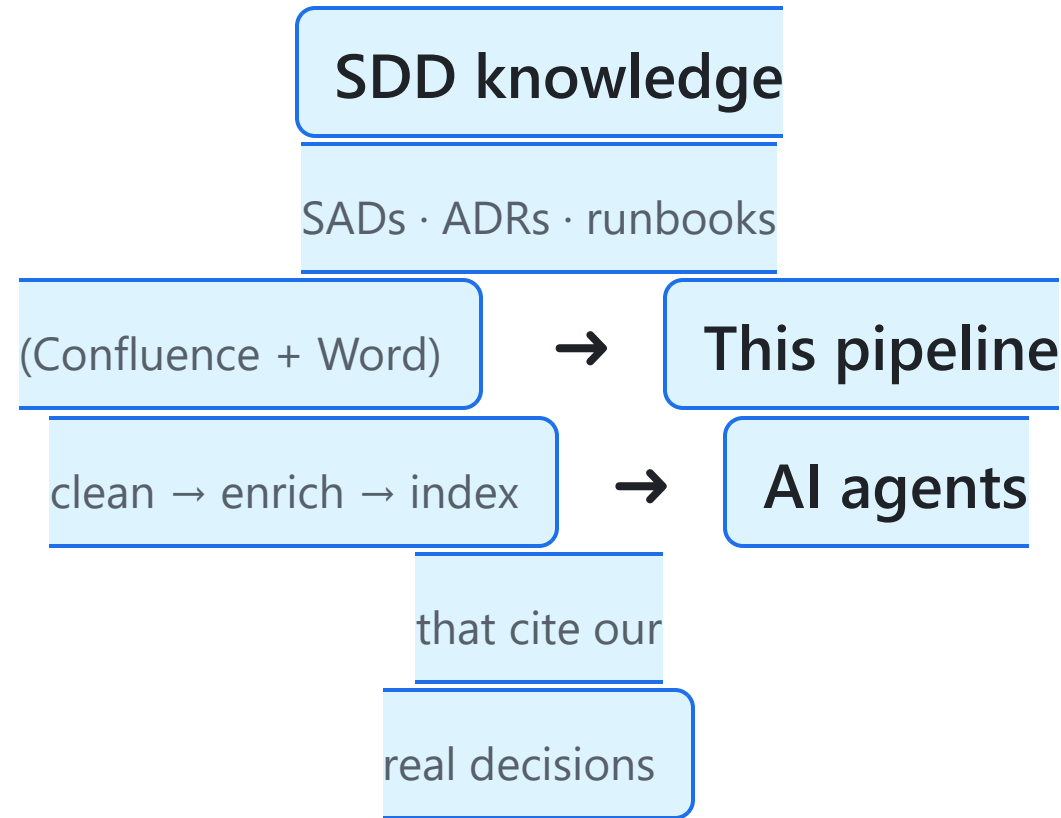
From SDD docs to an AI that *knows* them

Before → After: making our design knowledge usable by agents

Software Design Documents · semantic search · grounded GitHub Copilot

The goal — connect two things we already have

We **already** own both halves. They just can't talk to each other yet.



Scope: ingest our design-doc exports → turn them into one clean, searchable

Before → After

✗ Before

- SDD content **trapped** in Confluence/Word — macros, layout cruft
- Copilot writes ADRs with **zero grounding** → confident guesses
- New design doc = **re-read 5 old ones** by hand first
- Finding a past decision = keyword search + scrolling
- Onboarding leans on a senior engineer's memory

✓ After

- Confluence + Word → **one clean Markdown corpus**, provenance kept
- Agents **retrieve real, cited decisions** before they draft (RAG)
- Ask a question → get the **exact section** (semantic + filtered)
- Quality gates keep junk out, so agents ground on **good data**
- The corpus answers — **self-serve**, not a person

Why a smart AI still gets *our* architecture wrong

An LLM knows the public world — **not our internal decisions**. Ask *"how does our checkout saga compensate?"* and it returns a **confident guess**.

	Path	Result
AI alone	question → the model's memory	✗ plausible-sounding guess
AI + RAG	question → retrieve from our docs → answer	✓ grounded, cited answer

Why RAG is required: Retrieval-Augmented Generation injects **our current, real** knowledge at answer-time — cheaper than re-training the model, never goes stale, and every answer is **auditable** because it shows its sources.

RAG is only as good as what it retrieves

Retrieve fuzzy or wrong passages → the agent **still hallucinates**. So before indexing, the pipeline **enriches** every chunk — tagging *what it is* and *what it connects to*.

Plain chunk → weak retrieval	Enriched chunk → precise retrieval
just raw text	<code>type: decision · breadcrumb · entities: order.created · depends_on / exposes</code>

Why enrichment is required: it lets retrieval **filter to just the decisions** and answer cross-reference questions like "*which services consume `order.created` ?*" — the precision that makes RAG **trustworthy** on dense SDDs. Without it, the agent gets vague context and guesses anyway.

The benefits



Grounded agents

Copilot's ADR agent drafts from **our** decisions, not hallucinations — it cites the source doc.



Answers in seconds

Semantic search across the whole corpus, filterable to *just the decisions* — no more scrolling Confluence.



One clean source of truth

Confluence HTML *and* Word .docx → consistent Markdown with author/space/page kept as metadata.



Trustworthy & cheap to run

Quality gates block bad data from the index; most of it runs with **no AI model** — fast, in CI or a container.

It works today — and what we need

Proven now (real numbers from the repo):

- **Confluence + Word both convert** to clean Markdown, provenance intact.
- **5 real design docs → 107 enriched chunks in ~1 second** — with **no AI model** loaded.
- **664 automated tests passing** — built like a product, not a script.
- Grounding wired into **GitHub Copilot's ADR agent**; live demo runs in seconds.

The ask — a decision, not more code:

1. Choose the embedding engine — **self-hosted (free)** vs **Azure (no setup, per-call cost)**.
2. Give us the **full SDD corpus** to ingest. The pipeline is ready for it.

Want the architecture, robustness gates, and test detail? → technical appendix deck (`sdd-pipeline-deep-dive.md`).